Deep Learning Approaches for Medical Image Analysis: From Classification to Segmentation

Team 7: Siddartha Bandi, Sri Vamsi Chitta, Prabjyot Chavan

Abstract

Deep learning models used in medical image analysis are ResNet for classification, YOLO for detection, and U-Net for segmentation. The models have significantly enhanced accuracy and efficiency of disease and abnormality detection on a range of medical imaging modalities. Their use within the clinical environment improves diagnostic speed, accuracy, and result. Problems like lack of annotated data and class imbalance are also tackled, with focus placed on future directions for research to enhance model robustness and clinical usefulness.

Keywords Deep learning, medical image analysis, ResNet, YOLO, U-Net, classification, detection, segmentation, healthcare AI.

Citation Liu, X., Gao, K., Liu, B., Pan, C., Liang, K., Yan, L., Ma, J., He, F., Zhang, S., Pan, S., & Yu, Y. (2021). Advances in deep learning-based medical image analysis. *Science China Information Sciences, 64*(10), 211101. https://doi.org/10.1007/s11432-019-2790-1

Background

Medical imaging is today a fundamental component of medicine and a mandatory component of the diagnosis, monitoring, and planning of the treatment of many diseases, ranging from cancer to infectious disease. Analysis of images was once experience-based and image interpretation visual, and therefore time-consuming and potentially subjective.

Recent advances in artificial intelligence, especially deep learning, transformed this field. Deep architectures such as ResNet, YOLO, and U-Net facilitated automated interpretation and analysis of clinical images that improve speed, reproducibility, and accuracy for detecting abnormal findings. The methods allow image classification, object detection, and segmentation and provide robust, data-driven findings to clinicians for delivering better care to patients.

The confluence of AI algorithms and medical imaging equipment is leading in a new age of intelligent diagnostics, reducing workflow and bringing the potential of precision medicine

into view. Continued research and development of further developing more robust, explainable algorithms will likely continue to propel health provision over the next few years.

Summary of the Chosen Paper

The selected paper provides an in-depth overview of deep learning applications in medical image analysis. It covers how convolutional neural networks (CNNs) and their variants are used for four major tasks: classification, detection, segmentation, and registration. Key architectures highlighted include AlexNet, VGG, ResNet, DenseNet, U-Net, and nnU-Net, as well as detection frameworks like YOLO, SSD, and Faster R-CNN.

Applications demonstrated in the review include stroke lesion and hemorrhage detection, cardiac imaging and segmentation, liver lesion classification, and bone fracture recognition. Beyond successes, the paper outlines challenges such as limited annotated datasets, class imbalance in rare diseases, and the interpretability of models for clinical trust. To address these, the authors emphasize approaches like federated learning for data privacy, transfer learning for data efficiency, and the design of novel loss functions. These insights make the paper not only a survey of progress but also a roadmap for future research.

Algorithms and Approaches:

ResNet for Image Classification

Architecture:

ResNet (Residual Network) was a deep convolutional neural network that introduced the application of skip connections or residual blocks into the mix. These allow the model to "skip" one or more layers, effectively making gradients travel smoother backward through deep networks when training. This addresses the vanishing gradient issue that inhibits deep CNNs, allowing ResNet models (like ResNet-50) to be 50+ layers deep without a deterioration in training capability.

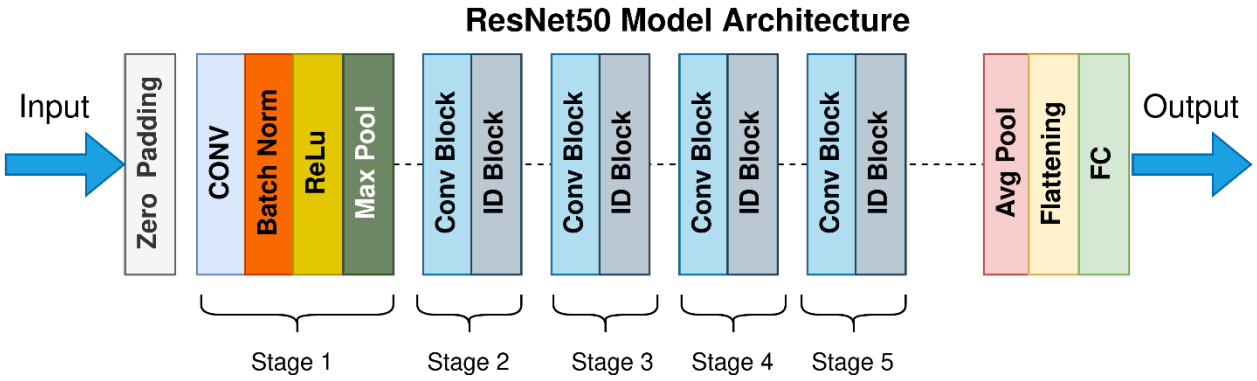


Figure 1: ResNet50 Model Architecture - This figure shows the high-level architecture of a ResNet model, illustrating the sequence of an initial convolution, a series of stacked residual blocks (Conv/ID Blocks), and a final classification layer.

How It Works:

ResNet uses stacked convolutional layers with batch normalization and Rectified Linear Unit (ReLU) activation. Skip connections use element-wise addition of output and input of stacked layers to help the network learn residual mappings (i.e., input vs. desired output difference) instead of direct mappings, which improves learning efficiency and accuracy.

Common Use Cases

ResNet is usually used in image classification tasks, where it classifies the whole image into a class. It may also be employed in medical imaging to classify presence or absence of disease on X-rays, MRIs, or CT scans (e.g., tumor vs. normal tissue detection).

Example: Brain MRI Classification

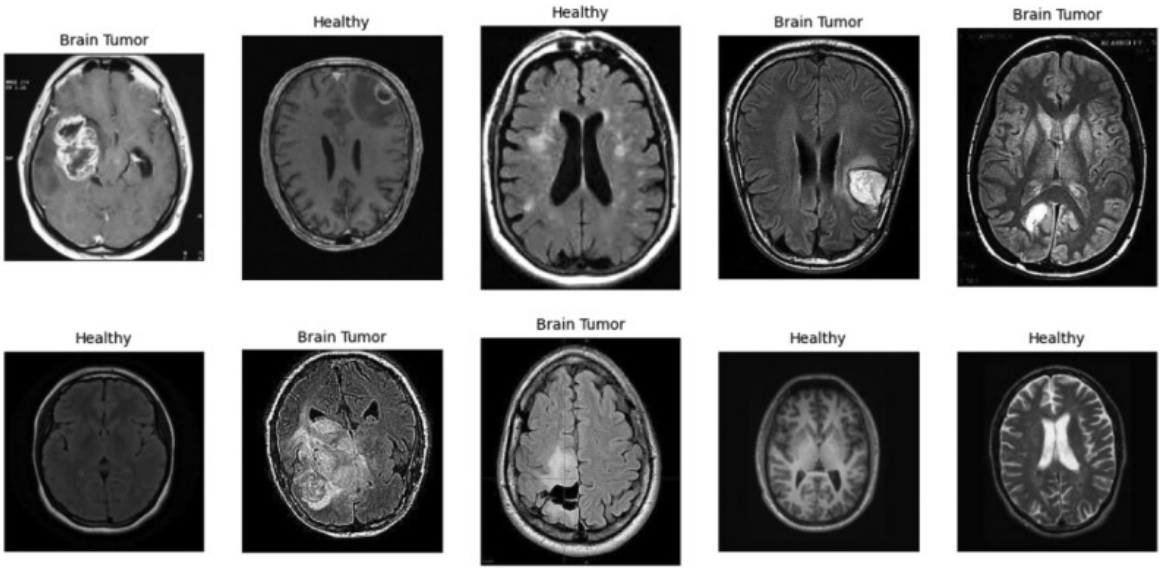


Figure 2: This image compares healthy brain MRIs with scans showing tumors. The scans labeled "Healthy" display normal brain anatomy, while the "Brain Tumor" images show irregular masses. Datasets like this are used to train deep learning models, such as ResNet, to automatically classify scans as either healthy or containing a tumor.

This image compares healthy brain MRIs with scans showing tumors. Deep learning models like ResNet can be trained on such datasets to automatically classify scans as *healthy* or *tumor*. Health images show normal anatomy, while tumor images display irregular masses.

Clinically, this approach supports radiologists by providing faster, more accurate diagnosis and reducing manual workload.

Pseudocode: ResNet Residual Block

```
function Residual_Block(input_data, output_channels):
  # 1. Save the original input for the skip connection.
  identity = input_data
  # --- Main Path ---
  # 2. First set of layers. Stride can be used to downsample the image.
  x = Conv_Layer(input_data, output_channels, stride=2)
  x = Batch_Norm(x)
  x = ReLU_Activation(x)
  # 3. Second set of layers.
  x = Conv_Layer(x, output_channels)
  x = Batch_Norm(x)
  # --- Skip Connection Path ---
  # 4. If the main path changed dimensions, resize the identity to match.
  if shape(identity) != shape(x):
    identity = Conv_1x1_Layer(identity, output_channels, stride=2)
  # --- Combine Paths ---
  # 5. Add the original (or resized) input back to the main path's output.
  output = Add(x, identity)
  # 6. Apply the final activation function.
  output = ReLU_Activation(output)
  return output
```

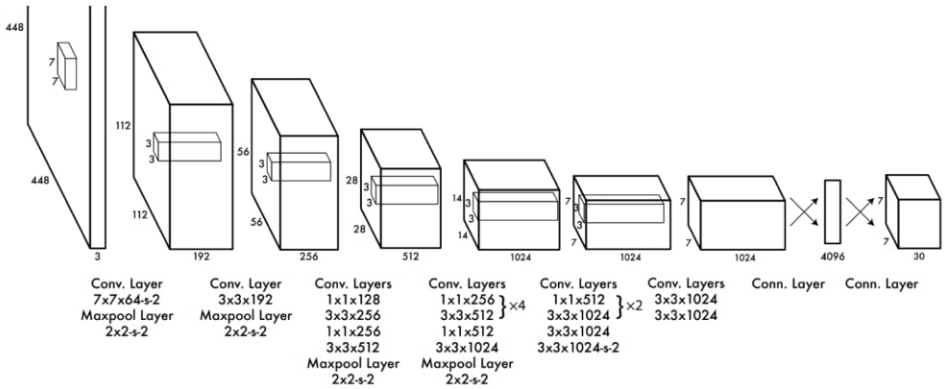
Implementation Explained

The implementation saves a copy of the block's input as the identity. This input is then passed through a main path of convolutional layers, batch normalization, and activations to learn complex features. If the main path alters the input's dimensions (e.g., with a stride), a simple 1x1 convolution is applied to the identity to make its shape match. Finally, the output of the main path and the resized identity are added together, and a final activation is applied. This "skip connection" is the core step that allows for the training of very deep networks

YOLO for Object Detection

Architecture:

YOLO (You Only Look Once) is a single-stage object detection network which processes the entire image in a single pass. It divides the image into a grid and predicts class probability and bounding boxes directly from each grid cell.



The Architecture. Our detection network has 24 convolutional layers followed by 2 fully connected layers. Alternating 1×1 convolutional layers reduce the features space from preceding layers. We pretrain the convolutional layers on the ImageNet classification task at half the resolution (224×224 input image) and then double the resolution for detection.

Figure 3: This figure shows the architecture for the YOLO (You Only Look Once) object detection model. The network consists of 24 convolutional layers followed by 2 fully connected layers and is designed to process an entire image in a single pass for real-time detection.

How It Works:

YOLO produces multiple bounding boxes per grid cell with class probabilities and confidence. There is a post-processing step called Non-Maximum Suppression (NMS), which removes redundant boxes to provide the final detections.

Typical Use Cases:

Typically used in real-time detection where performance and speed are crucial. In medical imaging, YOLO is used to find specific abnormalities like lung nodules, tumors, or fractures so that rapid triage or directions towards further examination can be enabled.

Example: Lung Nodule Detection in Chest X-rays

This image compares chest X-ray abnormality detection using different methods: input image, ground truth labels, expert reader, U-Net, and RetinaNet. Ground truth (B) and reader (C) show human-annotated lung abnormalities. U-Net (D) provides segmentation masks with confidence values, while RetinaNet (E) marks bounding boxes with detection confidence. The comparison demonstrates how AI models (U-Net for segmentation, RetinaNet for detection) approach the accuracy of human experts in locating chest abnormalities on medical images.

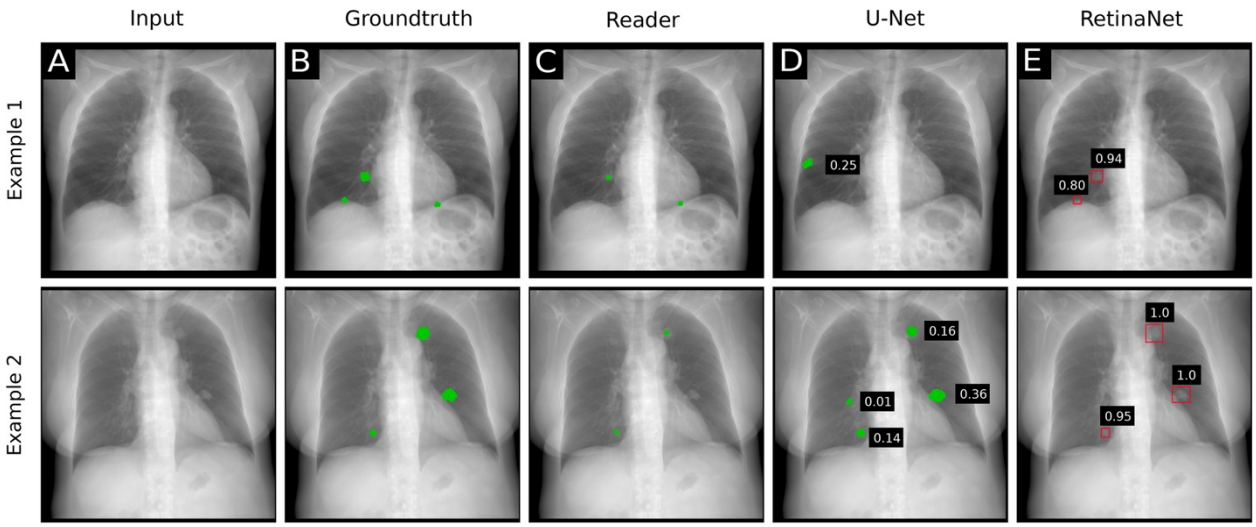


Figure 4: This image compares different methods for detecting abnormalities in chest X-rays. The panels show the original input image (A), the ground truth labels (B), annotations from an expert human reader (C), segmentation masks from a U-Net model (D), and detection bounding boxes from a RetinaNet model (E). The comparison demonstrates how AI models can approach the accuracy of human experts in locating chest abnormalities on medical images.

Pseudocode: YOLO Detection

```
function YOLO_Detect(image):
  # 1. Pass the entire image through the main CNN (e.g., Darknet).
  feature_map = CNN_Backbone(image)
  # 2. Divide the final feature map into an S x S grid.
  grid = divide_into_grid(feature_map)
  # 3. For each cell, predict bounding boxes and class probabilities.
  predictions = []
  for cell in grid:
    # Each cell predicts B boxes (x, y, w, h, confidence)
    # and C class probabilities.
    cell_prediction = model_head(cell)
    predictions.append(cell_prediction)
  # 4. Filter out boxes with low object confidence scores.
  confident_boxes = filter_low_confidence(predictions, threshold=0.5)
  # 5. Remove redundant, overlapping boxes for the same object.
  final_boxes = Non_Maximum_Suppression(confident_boxes)
  return final_boxes
```

Implementation Explained

The entire image is fed into a convolutional neural network one time to generate a feature map. This feature map is divided into a grid, where each grid cell is responsible for detecting objects, whose center falls within it. Each cell predicts a fixed number of bounding boxes and class probabilities. After prediction, boxes with a low confidence score are filtered out. Finally, a crucial post-processing step called Non-Maximum Suppression (NMS) is applied to remove redundant boxes that detect the same object, keeping only the best one.

U-Net for Image Segmentation

Architecture:

U-Net is a specialized convolutional neural network designed for biomedical image segmentation. Its architecture follows a symmetric encoder–decoder or 'U' shape. The encoder path compresses the image into abstract features, while the decoder path reconstructs spatial resolution. Crucially, skip connections link encoder and decoder layers, allowing the model to retain fine details lost during compression.

The strength of U-Net lies in its ability to work with relatively small datasets, which is common in medicine due to privacy constraints and annotation costs. In clinical practice, U-Net has been successfully applied to tasks like stroke lesion segmentation, cardiac

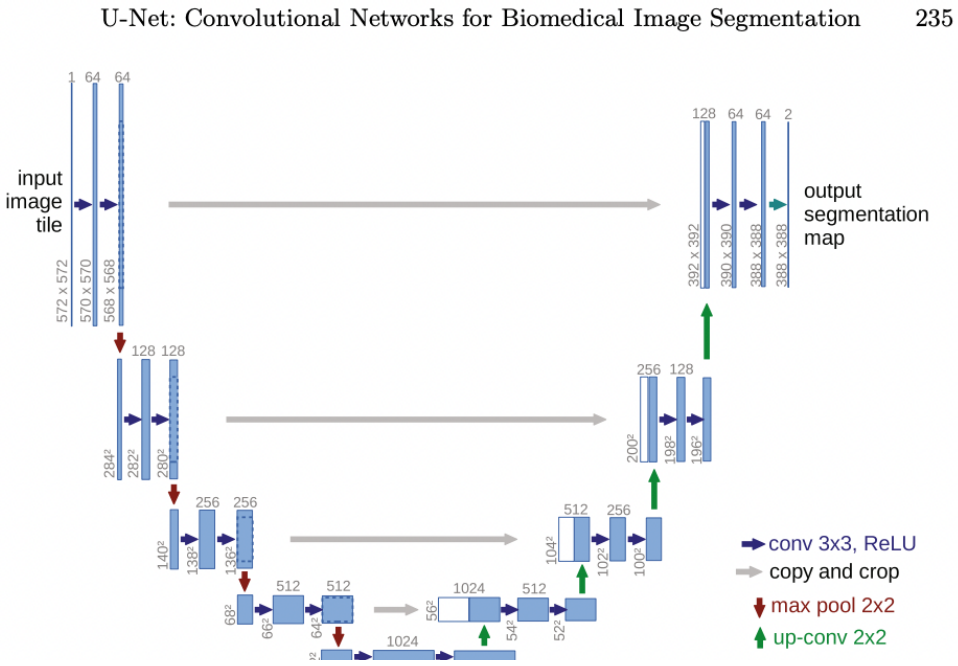


Figure 5: This figure shows the architecture of U-Net, a convolutional neural network designed for biomedical image segmentation. The model features a symmetric "U" shape with an encoder path (left side) that compresses the image and a decoder path (right side) that reconstructs it. The gray arrows indicate the crucial skip connections that link the encoder and decoder layers, allowing the model to retain fine details to produce a precise output segmentation map

chamber detection, and liver tumor outlining. Its precision at the pixel level makes it invaluable in treatment planning and surgical preparation, where boundaries must be accurately drawn.

How It Works:

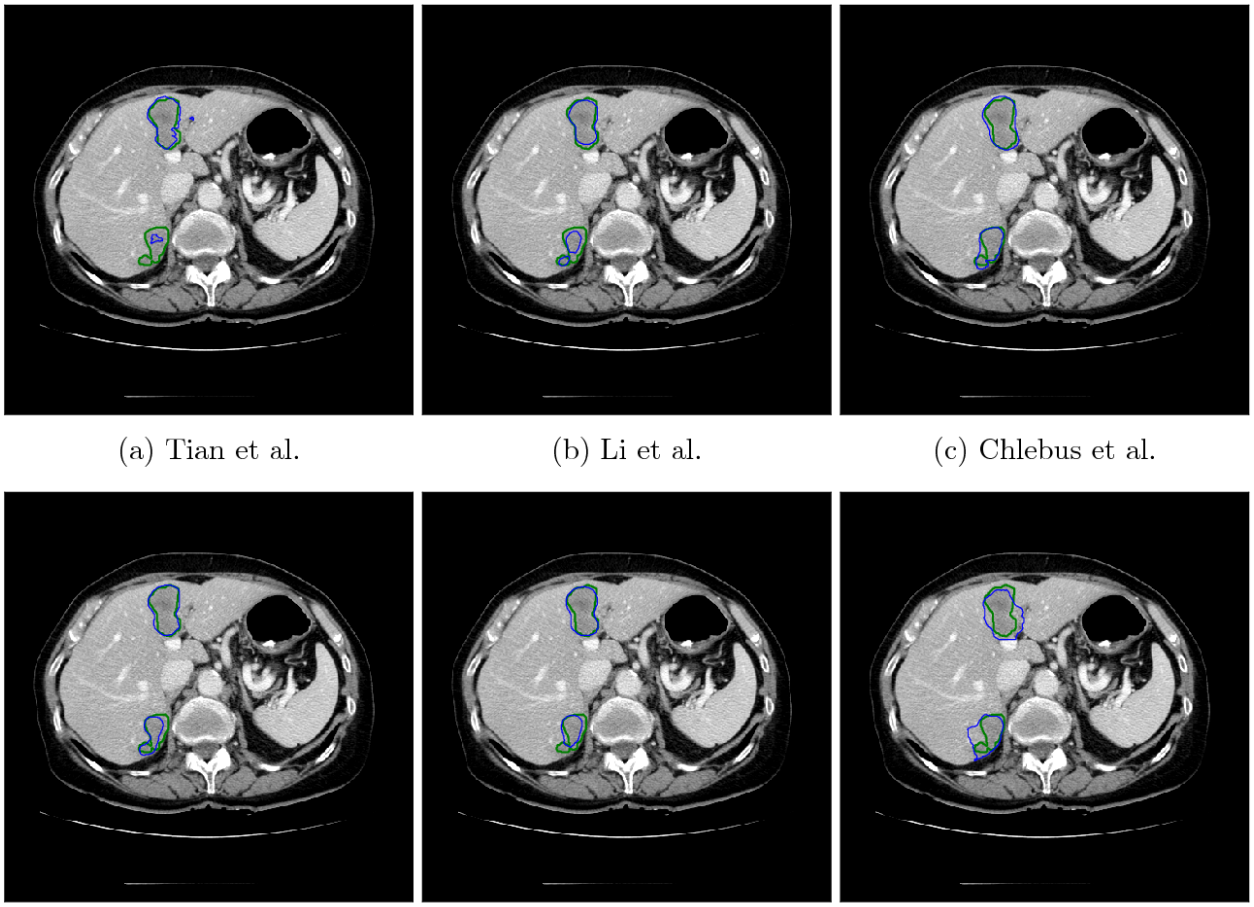
The encoder down samples and gets semantic information by applying convolution and pooling. The decoder up samples features maps and combines them with corresponding encoder outputs by using skip connections to preserve fine-grained spatial details. The final layer applies a SoftMax or sigmoid activation to provide a pixel-wise segmentation mask.

Common Use Cases:

Applied to tasks involving pixel-level segmentation especially medical image analysis, to segment tumors, lesions, or organs at high accuracy.

Example:

This figure shows a comparison of liver tumor segmentation results on CT images from six different research teams. Each sub-image displays a CT slice, with colored outlines indicating the segmented tumor regions produced by each team's algorithm. The visual comparison demonstrates how each method delineates tumors differently, helping evaluate segmentation accuracy and consistency across approaches for clinical applications.



(a) Tian et al. (b) Li et al. (c) Chlebus et al.

(d) Vorontsov et al. (e) Yuan et al. (f) Ma et al.

Figure 6: This figure shows a comparison of liver tumor segmentation results on CT images from six different research teams. Each sub-image displays a CT slice, with colored outlines indicating the segmented tumor regions produced by each team's algorithm. The visual comparison demonstrates how each method delineates tumors differently, helping evaluate segmentation accuracy and consistency across approaches for clinical applications.

Clinical Applications:

Deep learning techniques are employed to analyze medical images from all organ systems, including specific disease. For the nervous system, AI models segment and analyze brain structures to detect strokes, aneurysms, or tumors accurately in MRI scans, enhancing early treatment.

Cardiovascular applications are cardiac chamber segmentation and the detection of illnesses like heart valve abnormalities and coronary artery disease from ultrasound and CT scans. In the gastrointestinal tract, models segment and detect gastrointestinal tract lesions and liver tumors from CT and MRI images to guide treatment planning. For the

musculoskeletal system, AI aids in the detection of fractures and bone lesions from X-rays and CT scans to improve emergency diagnosis efficiency.

Each disease category now has top-of-the-line deep learning methods with very high accuracy, with benchmark dataset testing for performance comparison normalized. These applications show how AI can assist clinicians via automated, qualitative, and quantitative image interpretation with applicability to medical specialties.

Pseudocode: U-Net Segmentation

```
function U-Net_Segment(image):
  # --- Encoder (Contracting Path) ---
  # 1. Downsample and save outputs for skip connections.
  conv1 = Conv_Block(image)
  pool1 = Max_Pool(conv1)
  conv2 = Conv_Block(pool1)
  pool2 = Max_Pool(conv2)
  # ...continue down to the bottleneck...
  bottleneck = Conv_Block(pool_n)
  # --- Decoder (Expansive Path) ---
  # 2. Upsample and combine with skip connections.
  up1 = Upsample_Conv(bottleneck)
  concat1 = Concatenate(up1, conv2) # <-- Skip Connection
  decode1 = Conv_Block(concat1)
  up2 = Upsample_Conv(decode1)
  concat2 = Concatenate(up2, conv1) # <-- Skip Connection
  decode2 = Conv_Block(concat2)
  # ...continue up...
  # --- Final Layer ---
  # 3. Produce the final pixel-wise segmentation map.
  output_mask = Conv_1x1_Layer(decode2, num_classes=1)
  output_mask = Sigmoid_Activation(output_mask)
  return output_mask
```

Implementation Explained

The first half is the encoder, a contracting path that uses convolutions and max pooling to downsample the image, capturing contextual features. The output of each encoder block is saved. The second half is the decoder, an expansive path that uses up-convolutions to

upsample the feature maps back to the original image size. After each upsampling, the feature map is concatenated with the corresponding saved output from the encoder path. These skip connections are U-Net's key feature, re-introducing high-resolution details to achieve precise localization. A final 1x1 convolution and a sigmoid activation produce the final pixel-wise segmentation mask.

Discussion:

Together, U-Net, YOLO, and ResNet represent three pillars of medical image analysis: segmentation, detection, and classification. Each addresses a unique aspect of clinical workflow, and when combined, they create a comprehensive toolkit for AI-assisted diagnosis.

U-Net provides -> pixel-level precision ->, essential for mapping anatomical structures or lesions.
YOLO delivers ->speed and efficiency ->, allowing real-time localization of abnormalities.
ResNet ensures -> robust classification ->, enabling accurate differentiation of disease categories.

Despite these advances, challenges persist. Annotated datasets remain limited, particularly for rare diseases. Class imbalance skews performance toward more common conditions. Moreover, the 'black box' nature of these models raises concerns about interpretability. Solutions include federated learning, which allows collaborative training across hospitals without sharing raw data, and interpretable AI techniques like saliency maps, which help visualize decision-making processes. Addressing these challenges will be crucial for trustworthy clinical deployment.

Conclusion:

This project explored three cornerstone deep learning approaches in medical imaging: U-Net for segmentation, YOLO for detection, and ResNet for classification. Each algorithm demonstrated distinct strengths that contribute to healthcare innovation: U-Net excels at delineating complex anatomical regions, YOLO enables rapid detection of critical abnormalities, and ResNet delivers high-accuracy disease classification.

Our findings emphasize that while AI models have already achieved state-of-the-art performance in many tasks, further progress depends on addressing data limitations, class imbalance, and interpretability challenges. Future research directions include leveraging transfer learning, federated learning, and explainable AI frameworks to enhance both accuracy and trustworthiness. By focusing on both the mechanics of the models and their clinical impact, this project illustrates how AI is reshaping the landscape of healthcare diagnostics.

References:

1. He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep residual learning for image recognition*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 770–778. https://doi.org/10.1109/CVPR.2016.90

2. Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). *You Only Look Once: Unified, real-time object detection*. Proceedings of the IEEE Conference on Computer **Vision and** Pattern Recognition (CVPR), 779–788. https://doi.org/10.1109/CVPR.2016.91

3. Ronneberger, O., Fischer, P., & Brox, T. (2015). *U-Net: Convolutional networks for biomedical image segmentation*. International Conference on Medical Image Computing and Computer-Assisted Intervention (MICCAI), 234–241. https://doi.org/10.1007/978-3-319-24574-4_28

4. Litjens, G., Kooi, T., Bejnordi, B. E., Setio, A. A. A., Ciompi, F., Ghafoorian, M., ... & van der Laak, J. A. W. M. (2017). *A survey on deep learning in medical image analysis*. Medical Image Analysis, 42, 60–88. Medical Image Analysis, 3–11. https://arxiv.org/abs/1807.10165

5. Topol, E. J. (2019). High-performance medicine: the convergence of human and artificial intelligence. *Nature Medicine*, 25(1), 44–56. https://doi.org/10.1038/s41591-018-0300-7

6. Esteva, A., Robicquet, A., Ramsundar, B., Kuleshov, V., DePristo, M., Chou, K., ... & Dean, J. (2019). A guide to deep learning in healthcare. *Nature Medicine*, 25(1), 24–29. https://doi.org/10.1038/s41591-018-0316-z

7. Anwar, S. M., Majid, M., Qayyum, A., Awais, M., Alnowami, M., & Shabir, M. K. (2018). Medical Image Analysis using Convolutional Neural Networks: A Review. *Journal of Medical Systems*, 42(11), 226. https://doi.org/10.1007/s10916-018-1088-1